Sprint 1 & 2 Requirements Artifacts

Nov 8, 2026 · created by Cyn, edited and completed by Nathan Richardson, Aidan Prather, Dillon Figueredo, Maxwell Phachanla, Warren Tan, Fabrizio Gonzalez

Team 22: Nathan Richardson, Aidan Prather, Dillon Figueredo, Maxwell Phachanla, Warren Tan, Fabrizio Gonzalez, Cyn Tran

Project: Runway, a web app that turns uploaded bank CSVs into budgeting forecasts for people with variable income.

Each section below matches one row of the Requirements Stack by its ID, so every requirement has its own artifact. Each artifact lists the acceptance criteria being checked against, the evidence (screenshots or test cases), and who built and tested it. Test IDs such as TC-UPLOAD-3 refer to the Sprint 1 Testing Document in the GitHub repo.

Sprint	Requirement IDs	Story points
1 (revised)	1–10	43
2 (planned, due Nov 22)	11–18	50

Sprint 1 Artifacts (Requirements 1–10)

These replace the Sprint 1 artifacts from the last submission. Each requirement now has its own section, including the three that had no artifact before (4, 6 and 10). Figure numbers point to the screenshots under each section.

Req 1 – User Registration

Story points: 3 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

A new user can create an account with a username and password.

Acceptance criteria

1. The register page has Username, Password and Confirm Password fields.
2. A valid new username and matching passwords create the account and open the dashboard.
3. A username that already exists is rejected with an error message.
4. Passwords that do not match are rejected with an error message.
5. The password is saved as a hash, never as plain text.

Evidence: Figure 1 (registration page). Tests TC-USERREG-1, 2, 3 and TC-DB-1 all pass.

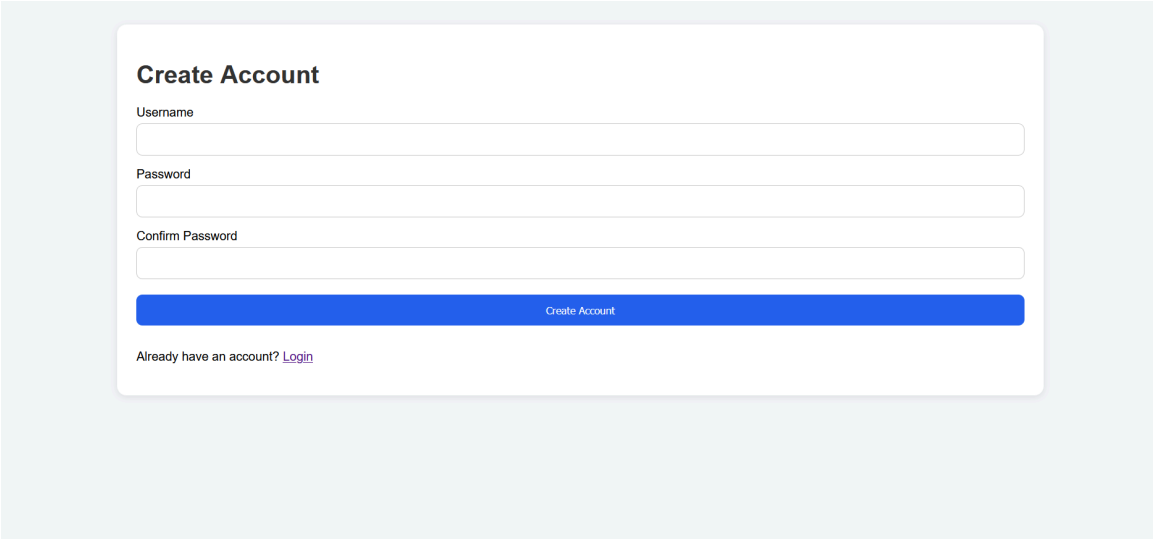


Figure 1: User registration page

Req 2 – User Login and Logout

Story points: 3 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

A registered user can log in to reach their dashboard and log out when done.

Acceptance criteria

1. A correct username and password open the user's dashboard.
2. An unknown username shows a login error.
3. A wrong password shows a login error.
4. Logout ends the session and returns to the login page.
5. The dashboard cannot be opened without logging in.

Evidence: Figure 2 (valid login), Figure 3 (dashboard), Figure 4 (logout). Tests TC-USERREG-4 to 7 pass.

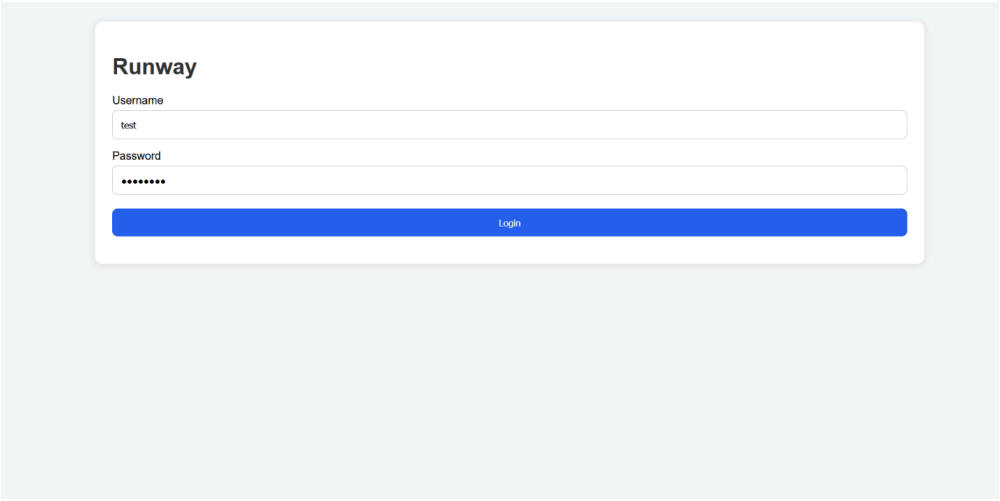


Figure 2: Valid account login

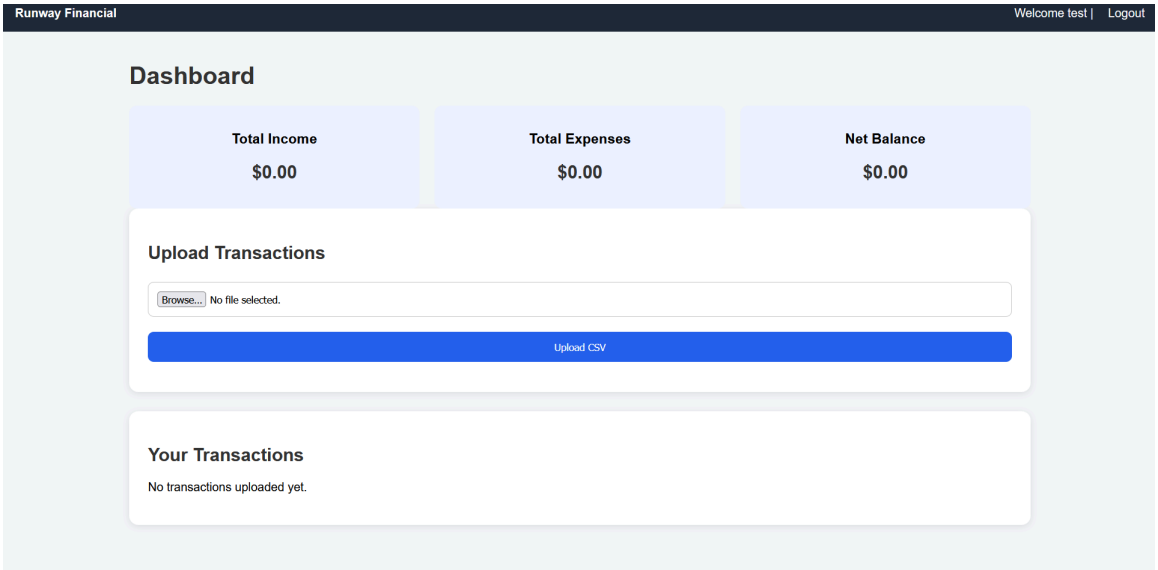


Figure 3: Dashboard after login

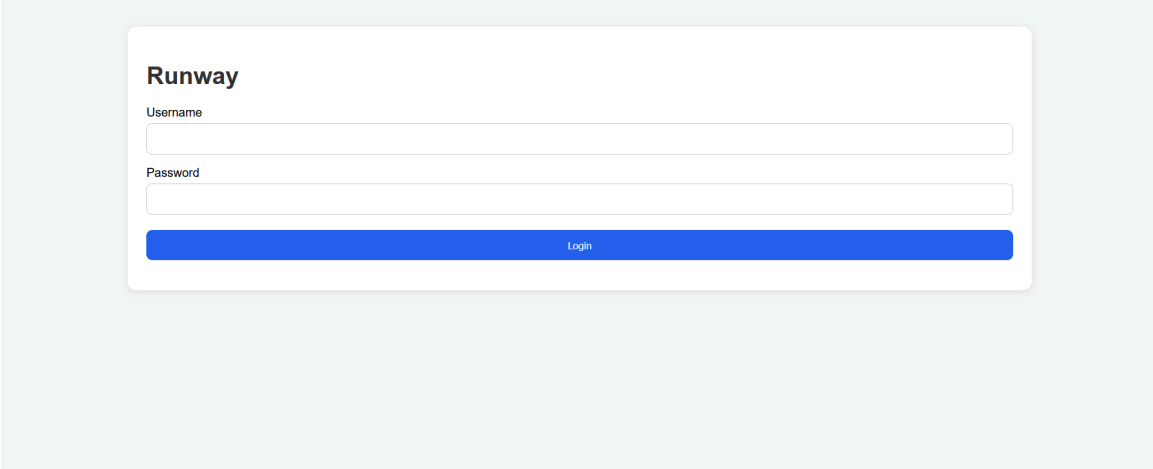


Figure 4: Pressing Logout returns to the login page

Known issue: TC-USERREG-8 fails. Pressing the browser Back button after logout still shows the cached dashboard page.

Req 3 – Password Reset Functionality

Story points: 3 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

A user who forgot their password can set a new one from the login page.

Acceptance criteria

1. The login page links to a password reset page.
2. Entering a registered username and two matching new passwords updates the password.
3. The user can then log in with the new password, and the old one stops working.
4. An unregistered username is rejected with an error message.
5. New passwords that do not match are rejected with an error message.

Evidence: Figure 5 (password reset page). Tests TC-ACCOUNT-1, 2, 3 pass.

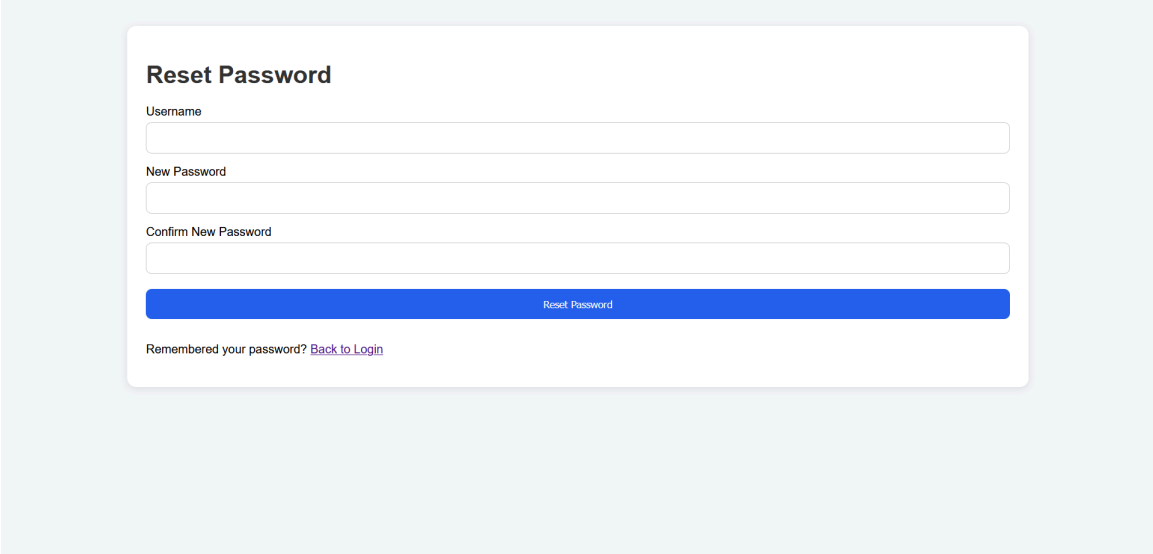


Figure 5: Password reset page

Req 4 – Testing of Account Management

Story points: 3 · **Tester:** Warren Tan

Requirements 1–3 were tested with 11 test cases. 10 passed and 1 failed. Full steps and results are in the Sprint 1 Testing Document.

Test ID	What it checks	Covers	Result
TC-USERREG-1	Register a new user	Req 1	Pass
TC-USERREG-2	Reject an existing username	Req 1	Pass
TC-USERREG-3	Reject mismatched passwords	Req 1	Pass
TC-USERREG-4	Log in with a valid account	Req 2	Pass
TC-USERREG-5	Reject an unknown username	Req 2	Pass
TC-USERREG-6	Reject a wrong password	Req 2	Pass
TC-USERREG-7	Log out to the login page	Req 2	Pass
TC-USERREG-8	Block the dashboard after logout (Back button)	Req 2	Fail
TC-ACCOUNT-1	Reset a password and log in with it	Req 3	Pass
TC-ACCOUNT-2	Reject reset for an unknown username	Req 3	Pass
TC-ACCOUNT-3	Reject mismatched reset passwords	Req 3	Pass

Req 5 – Upload Bank CSV

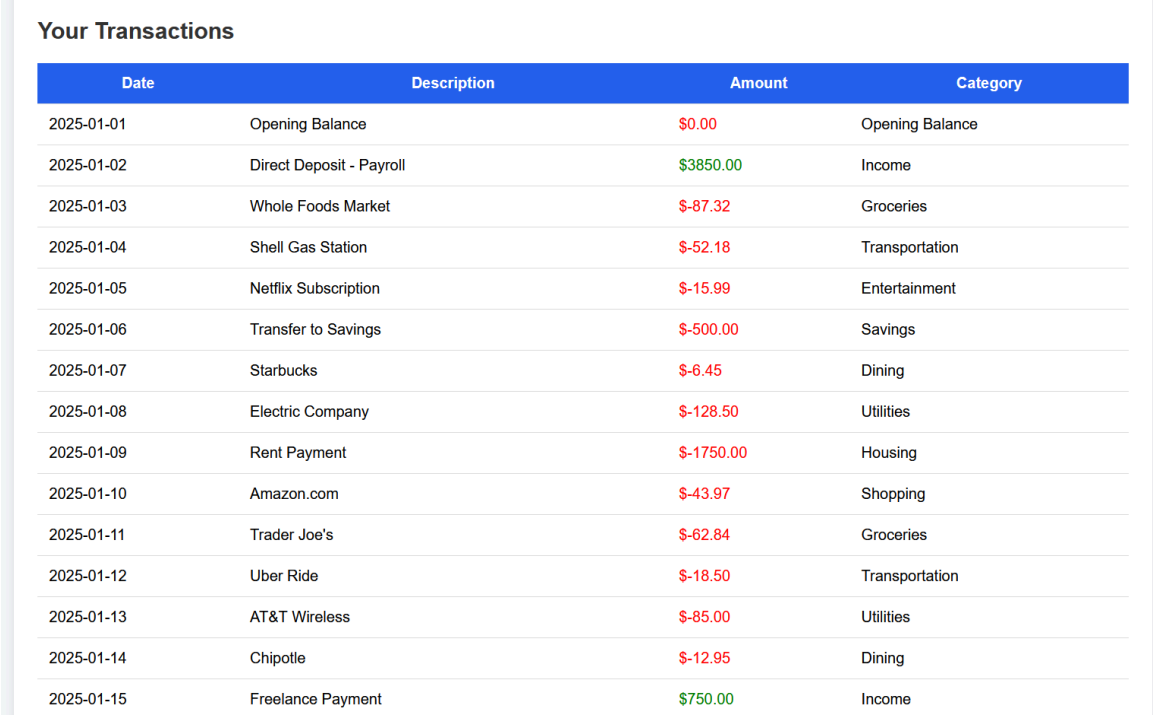
Story points: 5 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

A logged-in user can upload a bank statement CSV from the dashboard.

Acceptance criteria

1. The dashboard has a file picker and an Upload CSV button.
2. Uploading a valid CSV adds its transactions to the user's account and reloads the dashboard.
3. Clicking Upload CSV with no file selected is rejected with a message.
4. Only logged-in users can upload.

Evidence: Figure 6 (dashboard after uploading a sample CSV). Tests TC-UPLOAD-1 and TC-UPLOAD-2 pass.



Your Transactions

Date	Description	Amount	Category
2025-01-01	Opening Balance	$0.00	Opening Balance
2025-01-02	Direct Deposit - Payroll	$3850.00	Income
2025-01-03	Whole Foods Market	$-87.32	Groceries
2025-01-04	Shell Gas Station	$-52.18	Transportation
2025-01-05	Netflix Subscription	$-15.99	Entertainment
2025-01-06	Transfer to Savings	$-500.00	Savings
2025-01-07	Starbucks	$-6.45	Dining
2025-01-08	Electric Company	$-128.50	Utilities
2025-01-09	Rent Payment	$-1750.00	Housing
2025-01-10	Amazon.com	$-43.97	Shopping
2025-01-11	Trader Joe's	$-62.84	Groceries
2025-01-12	Uber Ride	$-18.50	Transportation
2025-01-13	AT&T Wireless	$-85.00	Utilities
2025-01-14	Chipotle	$-12.95	Dining
2025-01-15	Freelance Payment	$750.00	Income

Figure 6: Dashboard transaction table after uploading a sample CSV

Req 6 – Validate CSV Format

Story points: 5 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

The app checks that an uploaded file has the columns it needs before saving anything. Bank exports name their columns differently, so each column has accepted names (not case-sensitive):

Column	Required	Accepted names
Date	Yes	date, transaction date, posted date
Description	Yes	description, merchant, transaction description

Column	Required	Accepted names
Amount	Yes	amount, transaction amount
Category	No	category, expense category, type

Acceptance criteria

1. A CSV with any accepted name for each required column is accepted.
2. A CSV missing Date, Description or Amount is rejected with a message naming the missing column, such as "Amount column missing".
3. A rejected file saves no transactions.
4. The user stays on the dashboard after a rejection instead of seeing an error page.

Evidence: Test TC-UPLOAD-3 uses invalid_test_transactions.csv from the repo.

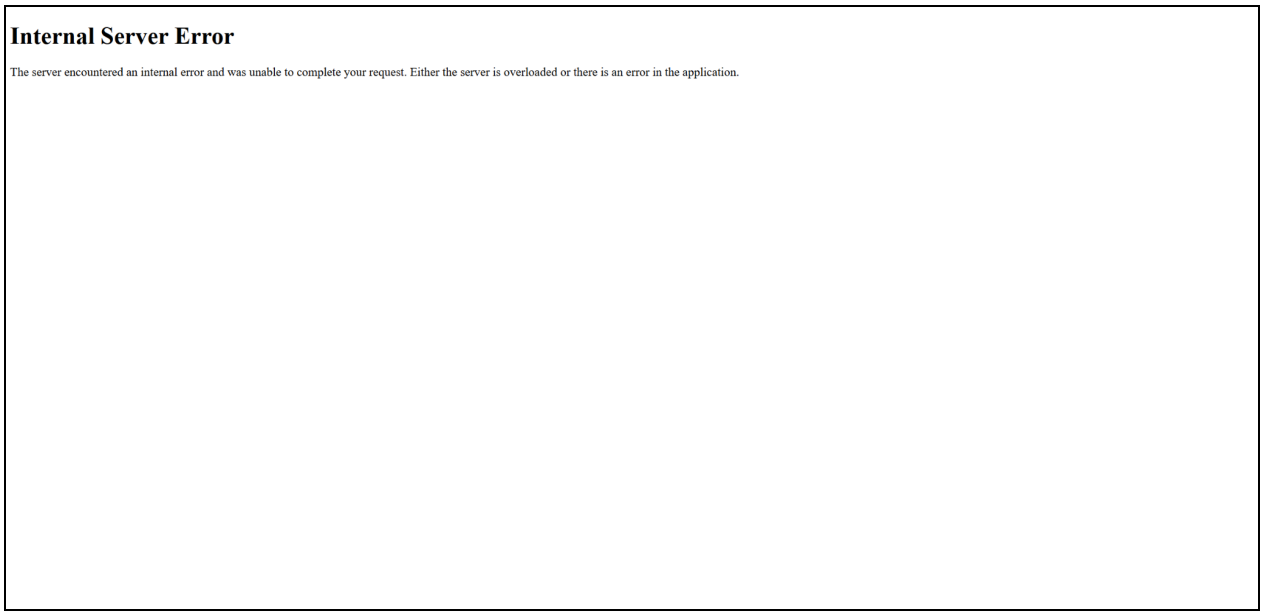


Figure 7: Dashboard output when user upload invalid csv file

Known issue: TC-UPLOAD-3 currently fails. The missing column is detected, but the error is not caught, so the user sees an exception page.

Req 7 – Store Transaction Data in Database

Story points: 8 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

Every parsed row is saved to PostgreSQL so it is still there at the next login.

Acceptance criteria

1. Each CSV row becomes one row in the transactions table with date, description, amount and category.
2. Each row stores the user_id of the account that uploaded it.
3. Transactions are still shown after logging out and back in.
4. A user only ever sees their own transactions.

Evidence: Figure 8 (transactions table in the database after uploading a CSV). Tests TC-DB-1 and TC-DB-2 pass.

"id"	"user_id"	"date"	"description"	"amount"	"category"
83	1	"2025-01-01"	"Opening Balance"	0	"Opening Balance"
84	1	"2025-01-02"	"Direct Deposit - Payroll"	3850	"Income"
85	1	"2025-01-03"	"Whole Foods Market"	-87.32	"Groceries"
86	1	"2025-01-04"	"Shell Gas Station"	-52.18	"Transportation"
87	1	"2025-01-05"	"Netflix Subscription"	-15.99	"Entertainment"
88	1	"2025-01-06"	"Transfer to Savings"	-500	"Savings"
89	1	"2025-01-07"	"Starbucks"	-6.45	"Dining"
90	1	"2025-01-08"	"Electric Company"	-128.5	"Utilities"
91	1	"2025-01-09"	"Rent Payment"	-1750	"Housing"
92	1	"2025-01-10"	"Amazon.com"	-43.97	"Shopping"
93	1	"2025-01-11"	"Trader Joe's"	-62.84	"Groceries"
94	1	"2025-01-12"	"Uber Ride"	-18.5	"Transportation"

Figure 8: First rows of the transactions table in PostgreSQL after uploading a CSV

Req 8 – Display Uploaded Transactions

Story points: 3 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

The dashboard lists the user's transactions in a table.

Acceptance criteria

1. The table shows Date, Description, Amount and Category for every transaction.
2. Negative amounts show as -$100.00, not $-100.00.
3. A new upload shows up in the table right away.

Evidence: Figure 9 (transaction table on the dashboard). Test TC-UPLOAD-4 passes.

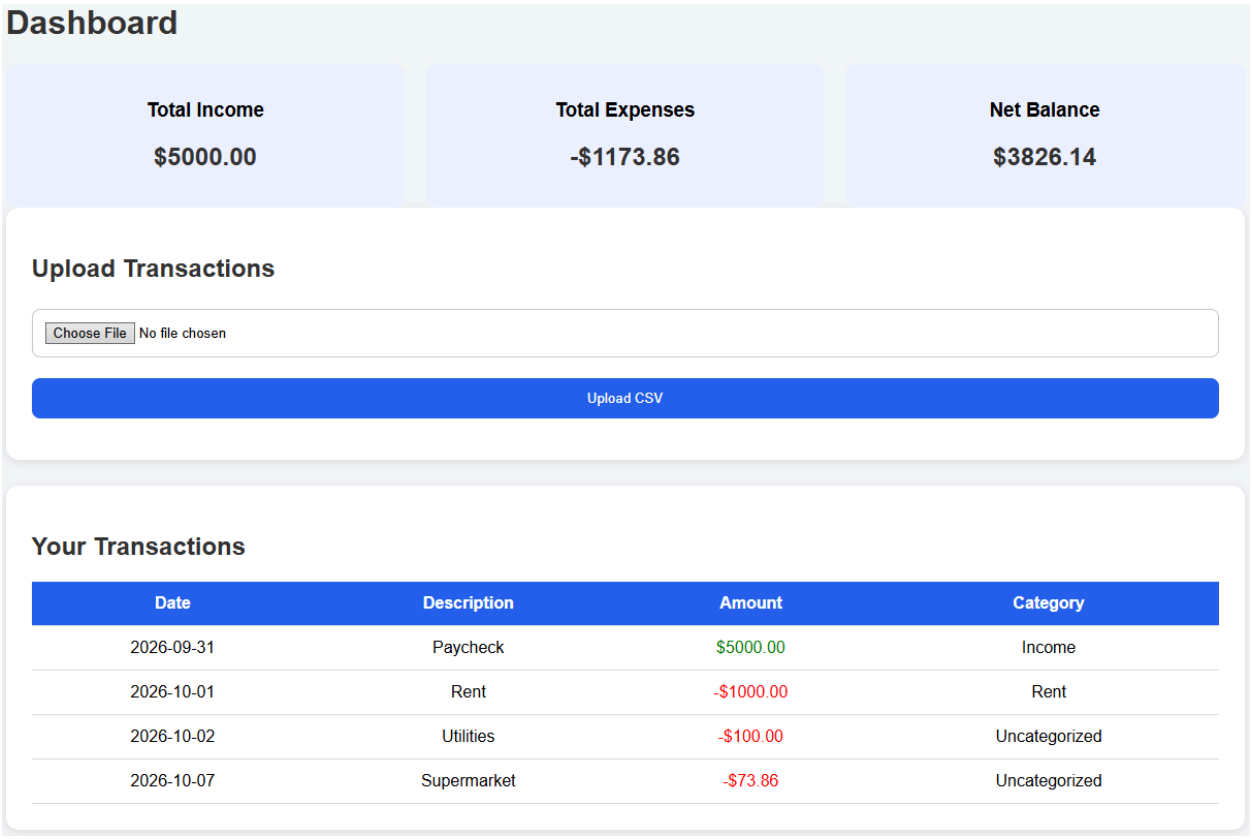


Figure 9: Display all user transaction

Req 9 – User Dashboard With Financial Summary

Story points: 5 · **Programmer:** Nathan Richardson · **Tested by:** Warren Tan

The dashboard shows a summary of the user's money above the transaction table.

Acceptance criteria

1. Income = sum of all positive amounts.
2. Expenses = sum of all negative amounts.
3. Net balance = income + expenses.
4. The number of transactions is shown.
5. With test_transactions.csv from the repo, the summary shows income $5,000.00, expenses -$1,173.86 and net balance $3,826.14.

Evidence: Figure 10 (financial summary after uploading a CSV). Test TC-UPLOAD-5 passes.

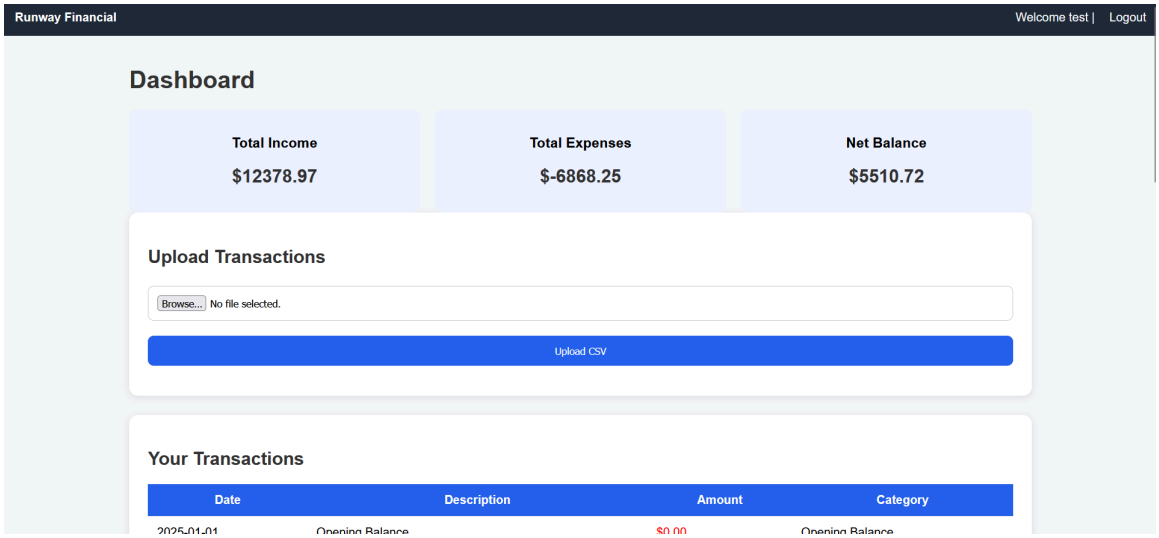


Figure 10: Financial summary after uploading a CSV

Req 10 – Testing of Bank CSV Upload and Display

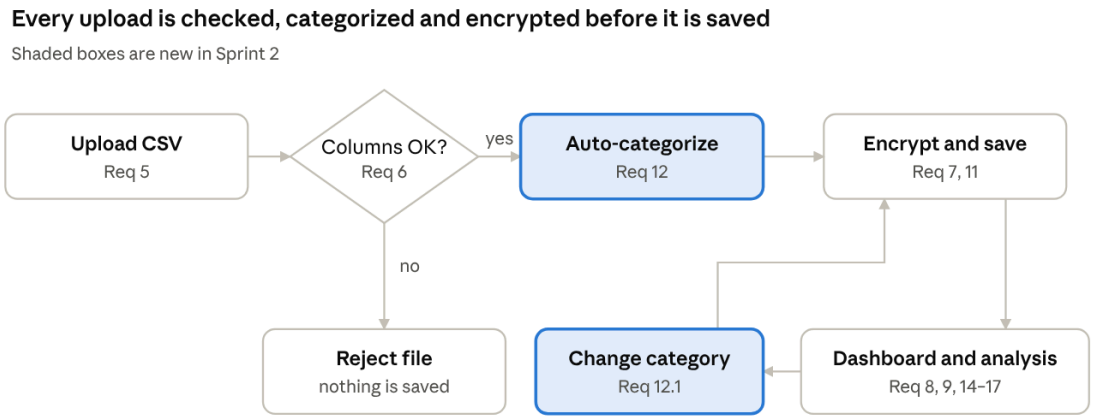
Story points: 5 · **Tester:** Warren Tan

Requirements 5–9 were tested with 7 test cases. 6 passed and 1 failed. Full steps and results are in the Sprint 1 Testing Document.

Test ID	What it checks	Covers	Result
TC-UPLOAD-1	Upload a valid transactions file	Req 5	Pass
TC-UPLOAD-2	Reject Upload with no file	Req 5	Pass
TC-UPLOAD-3	Reject an invalid transactions file	Req 6	Fail
TC-UPLOAD-4	Show the uploaded transactions	Req 8	Pass
TC-UPLOAD-5	Summary shows correct sums	Req 9	Pass
TC-DB-1	Users stored with hashed passwords	Req 1, 7	Pass
TC-DB-2	Transactions stored with user ID	Req 7	Pass

Sprint 2 Artifacts (Requirements 11–18)

Sprint 2 covers secure storage, transaction categorization and the monthly, recurring and seasonal analysis, for 50 story points. All of it works on the transactions saved in Sprint 1. Analysis results appear on a Spending Patterns page linked from the dashboard.



CSV upload flow · Sprint 1 and Sprint 2

A file missing a required column stops at Req 6. Every saved row is categorized and encrypted first, and the user can still fix its category later from the dashboard.

Req 11 – Secure Storage of Financial Data

Story points: 8 · **Programmer:** Aidan Prather · **Tested by:** Warren Tan

Transaction data is encrypted before it is written to the database, so someone reading the database directly cannot see a user's finances.

Design: The date, description, amount and category columns use Fernet encryption, which is symmetric encryption from Python's cryptography library. Values are encrypted on save and decrypted on load, so the rest of the app works with normal values. The key is read from FERNET_KEY in the .env file and is never committed to GitHub. A database migration encrypts transactions that were saved before this change.

Acceptance criteria

1. A SELECT on the transactions table shows unreadable encrypted bytes for date, description, amount and category.
2. The dashboard still shows the same readable values as before encryption.
3. Passwords stay stored as hashes.
4. The app gives a clear error if FERNET_KEY is missing or invalid, instead of saving unencrypted data.
5. Session cookies are set to HttpOnly and SameSite=Lax.

Verification: Screenshot of a SELECT on the transactions table next to the dashboard showing the same rows.

Req 12 – Automatic Transaction Categorization

Story points: 8 · **Programmer:** Maxwell Phachanla · **Tested by:** Warren Tan

Each uploaded transaction gets a category without the user doing anything.

Design: If the bank CSV already has a category, that one is used. Otherwise the description is matched against keyword lists in a fixed order, and the first match wins. Keywords match whole words, so "gas" does not match "Vegas". Money coming in with words like payroll or direct deposit is Income. Any other money coming in is also Income, and an expense with no match is Uncategorized.

Category	Example keywords
Rent	rent, apartment, landlord, leasing
Utilities	electric, water, internet, comcast, at&t
Subscriptions	netflix, spotify, hulu, adobe
Groceries	grocery, dillons, hy-vee, costco, aldi
Gas	quiktrip, shell, casey's, phillips 66
Dining	restaurant, starbucks, chipotle, doordash
Transportation	uber, lyft, parking, car wash
Shopping	amazon, ebay, best buy, etsy
Health	pharmacy, cvs, doctor, dental
Transfers	venmo, zelle, paypal, transfer

Acceptance criteria

1. "Apartments Rent" (-1000.00) is categorized as Rent.
2. "Dillons Grocery" (-59.00) is categorized as Groceries.
3. "Netflix" (-15.49) is categorized as Subscriptions.
4. "Payroll Direct Deposit" (+3200.00) is categorized as Income.
5. An expense with no matching keyword is categorized as Uncategorized.
6. A category column in the CSV is kept as-is.

Req 12.1 – Manual Transaction Categorization

Story points: 3 · **Programmer:** Maxwell Phachanla · **Tested by:** Warren Tan

The user can fix a wrong category from the dashboard.

Design: Each row of the transaction table has a dropdown with the 12 categories (the 10 above plus Income and Uncategorized). Changing it sends a POST to /transactions/<id>/category. The server only updates the transaction if it belongs to the logged-in user, then returns to the same row.

Acceptance criteria

1. Every transaction row has a category dropdown showing its current category.
2. Choosing a new category saves it, and it is still there after reloading or logging back in.
3. The page returns to the edited row, not the top.
4. Editing another user's transaction ID returns 404, and nothing changes.
5. A category that is not in the list returns 400, and nothing changes.

Req 13 – Testing of Transaction Categorization

Story points: 3 · **Tester:** Warren Tan

Requirements 12 and 12.1 will be tested with these cases and recorded in the Sprint 2 Testing Document.

Test ID	What it checks	Covers
TC-CAT-1	Known merchants get the right category (criteria 1–4 of Req 12)	Req 12
TC-CAT-2	Unmatched expense becomes Uncategorized	Req 12
TC-CAT-3	Category column in the CSV is kept	Req 12
TC-CAT-4	Change a category from the dropdown and reload	Req 12.1
TC-CAT-5	Changed category survives logout and login	Req 12.1
TC-CAT-6	Editing another user's transaction is blocked	Req 12.1

Req 14 – Monthly Income Analysis

Story points: 5 · **Programmer:** Dillon Figueredo · **Tested by:** Warren Tan

The user can see how much income came in each month and how much it varies. This matters because Runway is built for people whose pay changes month to month.

Design: Positive transactions are grouped by calendar month. A month in the range with no income counts as $0 instead of being skipped.

Acceptance criteria

1. A table lists total income for each month in the uploaded data, oldest to newest.
2. The average monthly income is shown.
3. The highest and lowest income months are named with their amounts.
4. Each month shows its change from the month before, in dollars and percent.
5. A month with no income shows $0.

Req 15 – Monthly Expense Analysis

Story points: 5 · **Programmer:** Dillon Figueredo · **Tested by:** Warren Tan

The user can see how much they spend each month and where it goes.

Design: Negative transactions are grouped by calendar month and by category, using the categories from Req 12 and 12.1.

Acceptance criteria

1. A table lists total expenses for each month, oldest to newest.
2. The average monthly expense is shown.
3. Each month breaks spending down by category.
4. Each month shows net cash flow (income minus expenses).
5. Changing a category with Req 12.1 updates the breakdown.

Req 16 – Identify Recurring Expenses

Story points: 5 · **Programmer:** Fabrizio Gonzalez · **Tested by:** Warren Tan

The app finds bills that repeat on a schedule, like rent or Netflix, and shows what they cost per month and per year.

Design: Descriptions are cleaned so "NETFLIX 1234" and "Netflix #5678" count as the same merchant. A merchant is recurring when all of these are true:

1. It has at least 3 charges.
2. The usual gap between charges fits one schedule: weekly (7 ± 2 days), biweekly (14 ± 3), monthly (30 ± 5), quarterly (91 ± 10) or yearly (365 ± 15).
3. At least 60% of the gaps fit that schedule.
4. The amount changes by no more than 35%. Within 10% it is labeled Fixed, otherwise Variable.

Acceptance criteria

1. The Spending Patterns page lists each recurring expense with its frequency, average amount, Fixed or Variable label, and next expected date.
2. Each item shows its monthly and annual cost, and the list is sorted by monthly cost, highest first.
3. Total recurring cost per month and per year is shown.
4. A merchant with only 2 charges, or with random gaps, is not listed.
5. Rows with unreadable dates are skipped, and the page says how many.

Req 17 – Detect Seasonal Income Patterns

Story points: 8 · **Programmer:** Fabrizio Gonzalez · **Tested by:** Warren Tan

The app shows which months and seasons bring more or less income than usual, so a seasonal worker can see slow periods coming. The same analysis is also shown for spending.

Design: Monthly totals are averaged by calendar month and by season (Winter = Dec–Feb, Spring = Mar–May, Summer = Jun–Aug, Fall = Sep–Nov). A seasonal index compares each one to the normal month: **seasonal index = average for that month or season / overall monthly average**

An index of 1.00 is a normal month. 1.15 or higher is labeled High, and 0.85 or lower is labeled Low.

Acceptance criteria

1. The Spending Patterns page shows a seasonal income section with an index and a High, Normal or Low label for each month and season.
2. Peak and low income months are listed by name.
3. The same breakdown is shown for expenses, including the top 6 spending categories and each one's peak season.
4. With less than 12 months of data, the page warns that a yearly pattern cannot be confirmed yet.
5. Months with no transactions count as $0, and the page warns about these gaps.

Req 18 – Testing of Recurring, Monthly and Seasonal Analysis

Story points: 5 · **Tester:** Warren Tan (with unit tests by Fabrizio Gonzalez)

Requirements 14–17 will be tested with these cases. The recurring and seasonal logic also has automated unit tests in the repo's tests/ folder.

Test ID	What it checks	Covers
TC-MONTH-1	Monthly income totals and average are correct	Req 14
TC-MONTH-2	Month with no income shows $0	Req 14
TC-MONTH-3	Monthly expense totals and category breakdown are correct	Req 15
TC-REC-1	Monthly bill with 3+ charges is listed with the right next date	Req 16
TC-REC-2	Merchant with only 2 charges is not listed	Req 16
TC-REC-3	Monthly and annual totals add up	Req 16
TC-SEAS-1	Month far above average is labeled High	Req 17
TC-SEAS-2	Less than 12 months shows the warning	Req 17
TC-SEAS-3	Seasonal income section is shown	Req 17